const, constexpr и символьные константы

До этого момента, все переменные, которые мы рассматривали, были *обычными*. Их значения можно было изменить в любое время, например:

```
int x { 4 }; // инициализация переменной x значением 4
x = 5; // изменяем значение x на 5
```

Тем не менее, иногда полезно использовать переменные, значения которых изменить нельзя — **константы**.

Оглавление:

1. Константы
2. Время компиляции и время выполнения
3. Спецификатор constexpr
4. Имена констант
5. Символьные константы
6. Использование символьных констант в программе

Константы

Возьмем к примеру величину силы тяжести на Земле: `9.8м/с^2`. Она вряд ли поменяется в ближайшее время. Использовать константу в этом случае будет наилучшим вариантом, так как мы предотвратим, таким образом, любое (даже случайное) изменение этого значения.

Чтобы сделать переменную константой — используйте **ключевое слово const** перед типом переменной или после него. Например:

```
const double gravity { 9.8 }; // предпочитаемый способ использовать const перед типом данных
int const sidesInSquare { 4 }; // но, по вкусу лучше - лучше
```

Несмотря на то, что язык C++ позволяет размещать `const` как перед типом данных, так и после него, хорошей практикой считается размещать `const` *перед* типом данных.

Константы должны быть инициализированы при объявлении. Изменить их значения с помощью операции присваивания нельзя:

```
const double gravity { 9.8 };
gravity = 9.9; // не допускается - ошибка компиляции
```

Объявление константы без её инициализации также вызовет ошибку компиляции:

```
const double gravity; // ошибка компиляции, константа должна быть инициализирована
```

Обратите внимание, константы могут быть инициализированы и с помощью неконстантных значений:

```
std::cout << "Enter your age: ";
int age;
std::cin >> age;

const int usersAge{age}; // в дальнейшем значение переменной usersAge не может быть изменено
```

Ключевое слово `const` является наиболее полезным (и наиболее часто используемым) с **параметрами функций**:

```
void printInteger(const int myValue)
{
    std::cout << myValue;
}
```

Таким образом, при вызове функции константа-параметр сообщает и гарантирует нам то, что функция не изменит значение переменной `myValue`.

Время выполнения и компиляции и время выполнения

Когда вы находитесь в процессе компиляции программы, то это **время компиляции** (англ. ***«compile time»***). Компилятор проверяет вашу программу на синтаксические ошибки и, если их нет, конвертирует код в объектные файлы.

Временной промежуток с момента старта выполнения программы и до момента окончания её работы называется **временем выполнения программы** (англ. ***«runtime»***). Код выполняется строка за строкой.

Спецификатор constexpr

В языке C++ есть два вида констант:

→ **Константы времени выполнения**. Их значения определяются только во время выполнения программы. Переменные типа `usersAge` и `myValue` выше являются константами времени выполнения, так как компилятор не может определить их значения во время компиляции. `usersAge` зависит от пользовательского ввода (который можно получить только во время выполнения программы), а `myValue` зависит от значения, переданного в функцию (это значение также определится только во время выполнения программы).

→ **Константы времени компиляции**. Их значения определяются во время компиляции программы. Например, переменная со значением силы тяжести на Земле является константой времени компиляции, так как мы её определяем во время написания программы (до начала её выполнения).

В большинстве случаев не важно какой тип константы вы используете: времени выполнения или времени компиляции. Однако, все же есть несколько ситуаций, когда C++ может потребовать константу времени компиляции вместо времени выполнения (например, при определении длины массива фиксированного размера — мы рассмотрим это несколько позже). Так как есть 2 типа констант, то компилятору нужно постоянно отслеживать, к какому из них относится какая переменная. Чтобы упростить это задание, в C++11 добавили **спецификатор constexpr**, который сообщает компилятору, что текущая переменная является константой времени компиляции:

```
constexpr double gravity {9.8}; // ок, значение определяется во время компиляции программы
constexpr int sum = 4 + 5; // ок, результат выражения 4 + 5 определяется во время компиляции программы

std::cout << "Enter your age: ";
int age;
std::cin >> age;
constexpr int myAge = age; // неправильно, переменная age не определяется во время компиляции программы
```

Использовать его вы, скорее всего, не будете, но знать о нем не помешает.

Правило: Любая переменная, которая не должна изменять свое значение после инициализации, должна быть объявлена с помощью спецификатора const (или constexpr).

Имена констант

Некоторые программисты пишут имена констант заглавными буквами. Другие используют обычные имена, только с префиксом `k`. Мы же не будем их как-то выделять, так как константы — это те же обычные переменные, просто с фиксированными значениями, вот и всё. Особой причины их выделять — нет. Однако, это дело привычки.

Символьные константы

На предыдущем уроке мы говорили о **магических числах** — литералы, которые используются в программе в виде констант. «Поскольку использовать их не рекомендуется, то какой выход?» — спросите вы. А я отвечу: «Использовать символьные константы». **Символьная константа** — это тот же литерал (магическое число), только с идентификатором. Есть 2 способа объявления символьных констант в языке C++. Первый способ хороший, а второй — не очень. Рассмотрим сначала плохой способ.

Плохой способ: Использовать макросы-объекты с текст_замена в качестве символьных констант.

Раньше этот способ широко использовался, так что вы все еще можете его увидеть в старых программах.

Как мы уже знаем, макросы-объекты имеют две формы: с `текст_замена` и без `текст_замена`. Рассмотрим первый вариант с `текст_замена`. Он выглядит следующим образом:

`#define идентификатор текст_замена`

Как только препроцессор встретит эту директиву, все дальнейшие появления `идентификатор` будут заменены на `текст_замена`. `идентификатор` обычно пишется заглавными буквами с нижним подчёркиванием вместо пробелов.

Например:

```
#define MAX_STUDENTS_PER_CLASS 30

//...
int max_students = numClassrooms * MAX_STUDENTS_PER_CLASS;
//...
```

Во время компиляции программы, препроцессор заменит все идентификаторы `MAX_STUDENTS_PER_CLASS` на литерал `30`.

Согласитесь, это гораздо лучше, нежели использовать магические числа, как минимум, по нескольким причинам. `MAX_STUDENTS_PER_CLASS` дает понимание того, что это за значение и зачем оно используется (это понятно даже без **комментариев**). Во-вторых, если число нужно будет изменить — достаточно будет внести правки только в директиву #define, все остальные идентификаторы `MAX_STUDENTS_PER_CLASS` в программе будут автоматически заменены новым значением при повторной компиляции.

Рассмотрим еще один пример:

```
#define MAX_STUDENTS_PER_CLASS 30
#define MAX_NAME_LENGTH 30

int max_students = numClassrooms * MAX_STUDENTS_PER_CLASS;
setMax(MAX_NAME_LENGTH);
```

Здесь понятно, что `MAX_STUDENTS_PER_CLASS` и `MAX_NAME_LENGTH` не являются одним и тем же объектом, хоть и имеют одинаковые значения.

Так почему же этот способ плохой? На это есть две причины:

→ Во-первых, макросы обрабатываются препроцессором, который заменяет идентификаторы на определенные значения. Эти значения не отображаются в отладчике (во время **отладки вашей программы**). При компиляции `int max_students = numClassrooms * 30;` в отладчике вы увидите `int max_students = numClassrooms * MAX_STUDENTS_PER_CLASS;`. «А как тогда узнать значение `MAX_STUDENTS_PER_CLASS`?» — спросите вы. А я отвечу: «Вам придется самостоятельно найти это в коде». А процесс поиска может занять некоторое время (в зависимости от размеров вашей программы).

→ Во-вторых, эти директивы всегда имеют глобальную область видимости (о ней мы поговорим позже). Это означает, что значения #define в одной части кода могут конфликтовать со значениями #define в другой части кода.

Правило: Не используйте директиву #define для создания символьных констант.

Хороший способ: Использовать переменные со спецификатором const.

Например:

```
const int maxStudentsPerClass { 30 };
const int maxNameLength { 30 };
```

Такие значения отображаются в отладчике, а также следуют всем правилам обычных переменных (в том числе и по области видимости).

Правило: Используйте спецификатор const для создания символьных констант.

Использование символьных констант в программе

Во многих программах символьные константы используются часто (в разных местах кода). Это могут быть физические или математические константы (например, число Пи или число Авогадро), или специфические значения в вашей программе. Чтобы не писать их каждый раз, когда они необходимы — просто определите их в одном месте и используйте везде, где они нужны. Таким образом, если вам придется изменить их значения — достаточно будет зайти в один файл и внести в него правки, а не искать эти константы по всей программе.

Алгоритм использования символьных констант в вашей программе:

→ *Шаг №1*: Создайте **заголовочный файл** для хранения констант.

→ *Шаг №2*: В заголовочном файле **объявите пространство имен**.

→ *Шаг №3*: Добавьте все ваши константы в созданное пространство имен (убедитесь, что все константы имеют спецификатор `const`).

→ *Шаг №4*: #include заголовочный файл везде, где нужны константы.

Например, файл constants.h:

```
#ifndef CONSTANTS_H
#define CONSTANTS_H

// Определяем пространство имен для хранения констант
namespace constants
{
    const double pi{3.14159};
    const double avogadro{6.0221413e23};
    const double my_gravity{9.2};
    // ... другие константы
}
#endif
```

Используйте оператор разрешения области видимости `::` для доступа к константам в файлах .cpp:

```
#include "constants.h"

//...
double circumference = 2 * radius * constants::pi;
//...
```

Если в программе много констант разных типов, то сделайте отдельные файлы для каждого из этих типов. Таким образом, отделив математические константы от специфических значений (которые могут быть разными в разных программах), вы сможете подключать файл с математическими константами к любой другой программе.